

How to get reproducible computational experiments



Thanks

Prelim committee: Daniel S. Katz, Darko Marinov, Reed Milewicz, Bertram Ludäscher

Thesis advisors: Daniel S. Katz, Darko Marinov

Sandia Mentor: Reed Milewicz

Consulting expert: Bertram Ludäscher

Takeaways

1. $\exists$ valuable but unexplored ways to improve reproducibility
2. It is feasible to research those for a 12-month thesis.

Background



1. Background

- 1. Definitions

- 2. Why reproducibility

2. Characterization of reproducibility in practice

3. Technical approaches to automatic reproducibility

4. Proposed work

5. Conclusion

Defining reproducibility

Reproducibility $\coloneqq$ The measurement can be obtained with stated precision by a **different team** using the **same measurement procedure**, the **same measuring system**, under the **same operating conditions**, in the same or a different location on multiple trials.

— (ACM Inc. staff 2020)

Related terms

	Repeatability	Reproducibility	Replicability
Experimental setup	Same	Same	Diff
Team	Same	Diff	Diff

Experimental setup := measurement procedure, measuring system, operating conditions

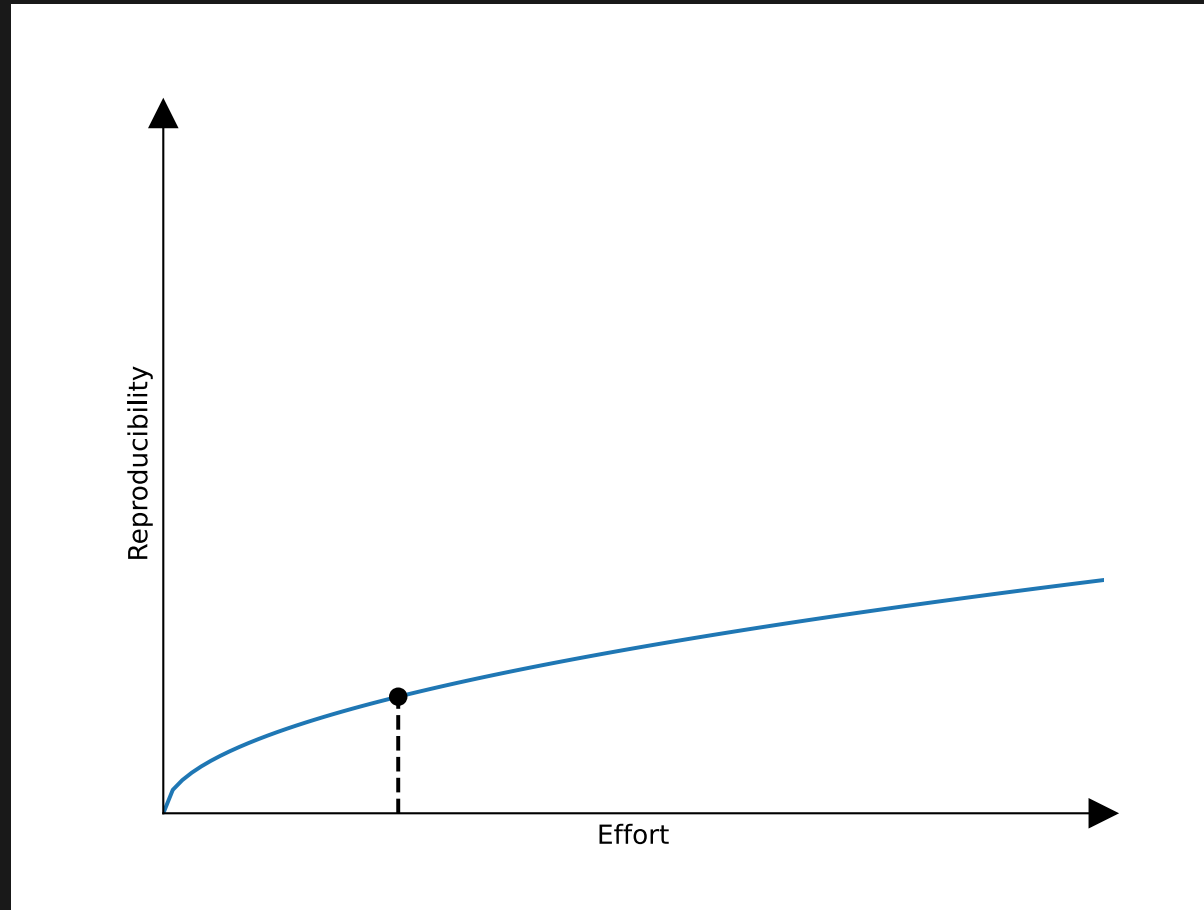
Why reproducibility

- Scientific claims should be scrutinizable
- Science builds off of other science
- Scientific research aims to impact practice
- Citation count ([Hitchcock 2004](#))

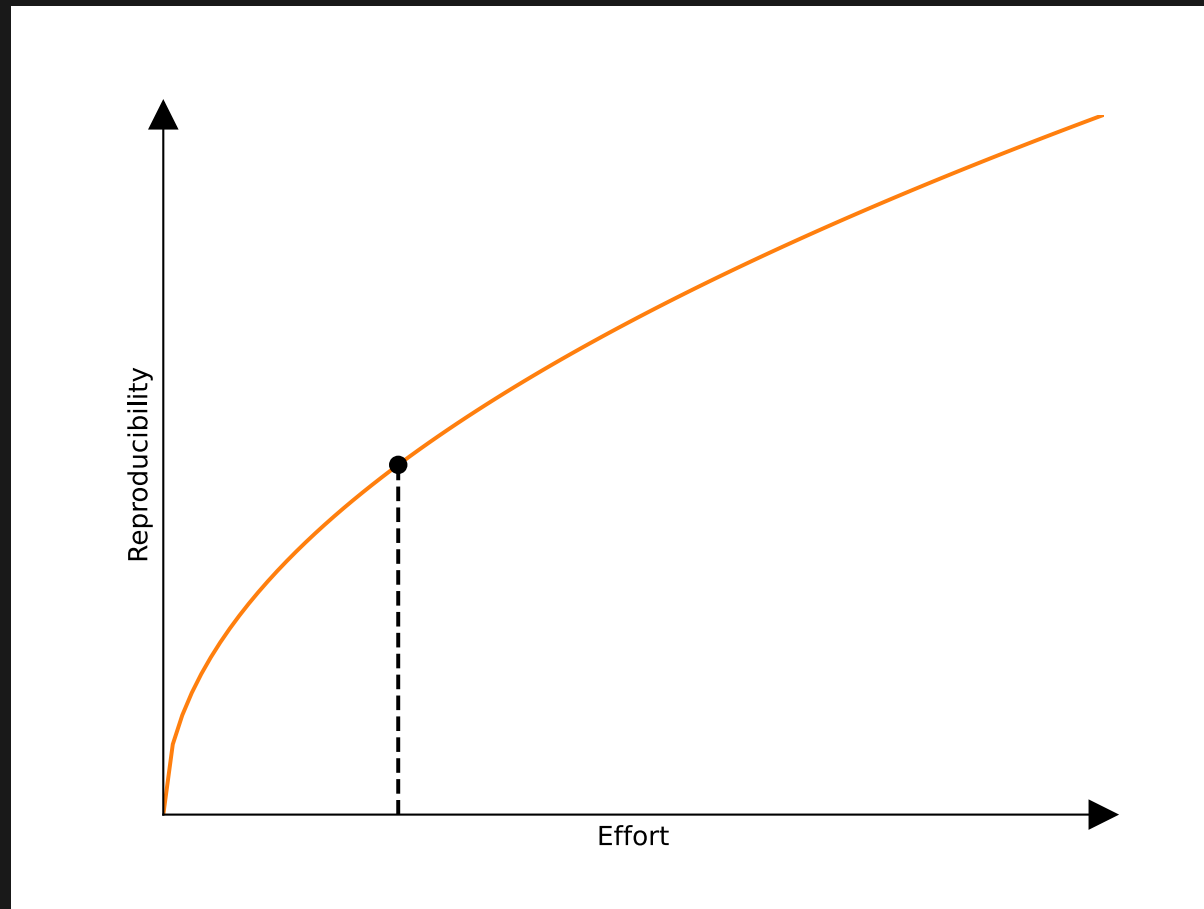


Robert K. Merton

Tradeoff between effort and reproducibility



Don't spend more effort; improve the techniques so
your effort goes further



Problem: Reproducibility is important for science but too costly to pursue sufficiently.

Characterization of reproducibility in practice



1. Background

2. **Characterization of reproducibility in practice**

1. Conclusions from prior work

2. Reproducibility in workflows

3. Technical approaches to automatic reproducibility

4. Proposed work

5. Conclusion

Why characterize reproducibility?

Is reproducibility a problem? How bad is it?

Has it improved over time or with tools?

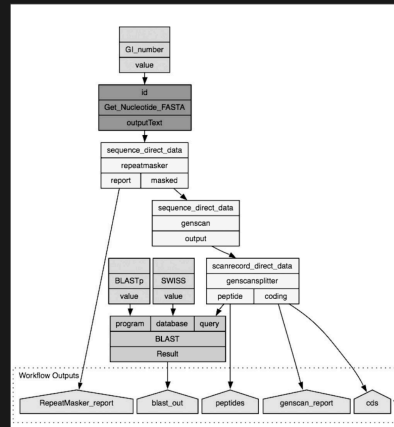
Focus on the most frequent failure modes

Conclusions from prior work

- Source is a significant problem (C, Gu, V)
- Common reason: labor to distribute and support (C, Go)
- Even once src is available, it might not build/run (C, I, Wa)
- Common reason: environment underspecified(Wa, I, Z)
- Even when runnable, result may be different due to platform differences (D, K, P, N)
- Collberg and Proebsting 2016, Di Tomaso et al. 2017, Gomes et al. 2022, Guunderson and Kjensmo 2018, Krafczyk et al. 2019, Neupane et al. 2019, Pimentel et al. 2019, Trisovic et al. 2021, Vandewalle et al. 2009, Wang et al. 2021, Zhao et al. 2012,

Reproducibility in workflows

- Workflow := language for digraph of coarse-grained tasks
- Orchestration helps eliminate "what to run? how to run it?" problems



Grayson, Marinov, Katz, Milewicz. *Automatic
Reproduction of Workflows in the Snakemake Workflow
Catalog and nf-core Registries.*

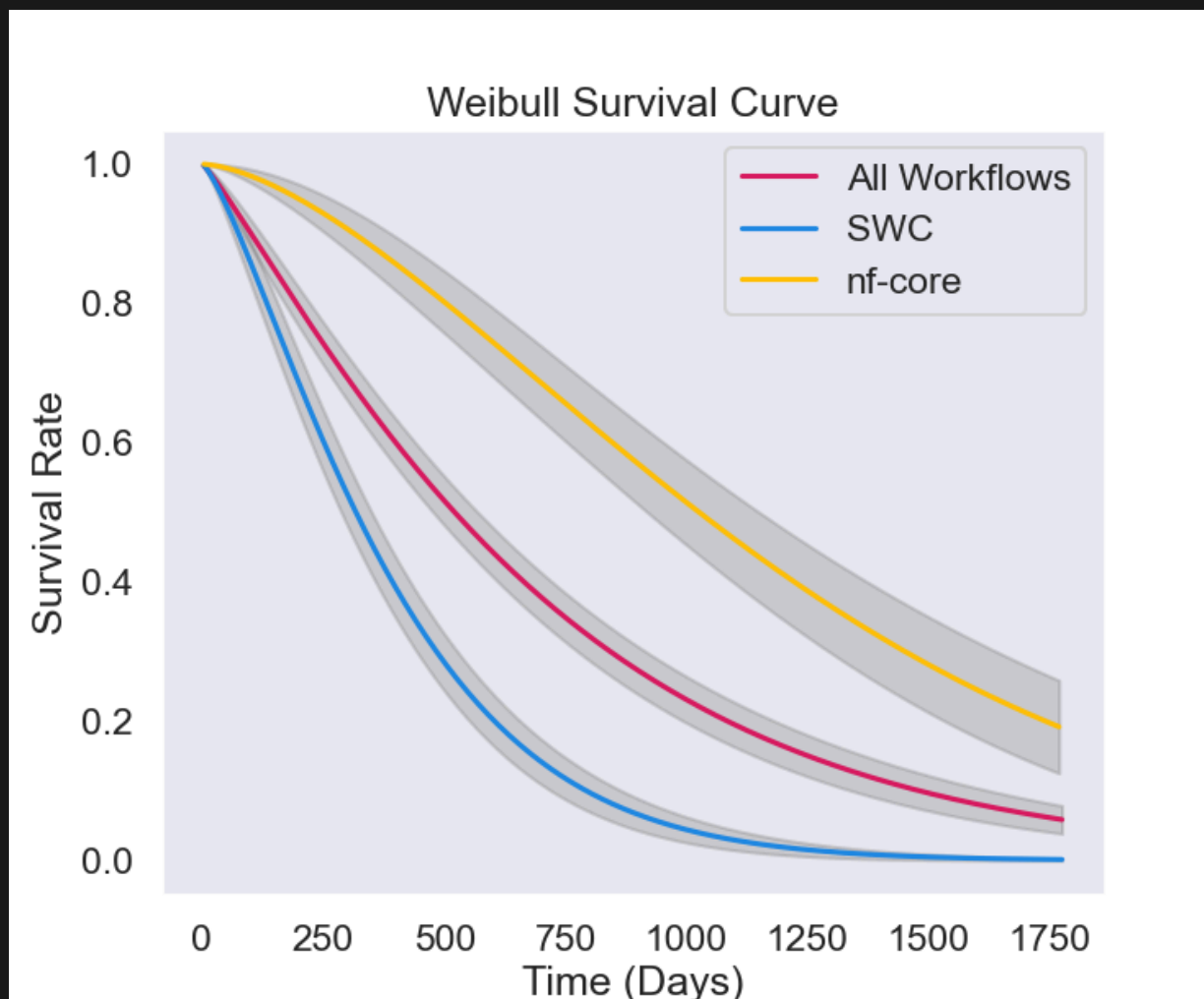
ACM REP '23 [10.1145/3589806.3600037](https://doi.org/10.1145/3589806.3600037)

- [Workflows.community](#) → workflow corpora
- [Snakemake](#) (2012 – present) and [Nextflow](#) (2013 – present)
- Each node can be containerized or Conda-sandboxed
- Should be better than arbitrary programs

Distribution of error modes for Snakemake and Nextflow workflows in Grayson et al. 2023.

Kind of crash	All	SWC	nf-core
Missing input	32.2%	43.8%	16.7%
Conda environment unsolvable	10.8%	18.9%	0.0%
Unclassified reason	7.9%	12.0%	2.4%
Timeout reached	7.0%	5.7%	8.8%
Singularity error	6.0%	6.6%	5.2%
Other (workflow script)	5.7%	1.5%	11.2%
Other (containerized task)	1.2%	0.0%	2.8%
Network resource changed	0.7%	0.0%	1.6%
Missing dependency	0.5%	0.9%	0.0%
No crash	28.1%	10.5%	51.4%
Total	100%	100%	100%

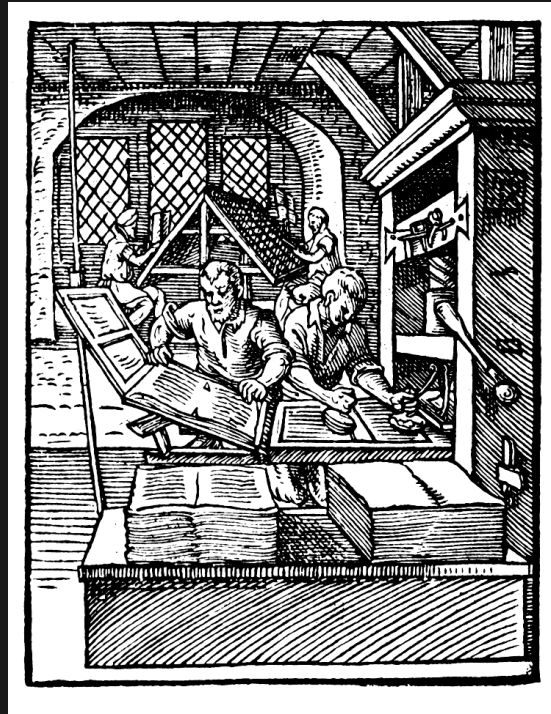
Survival analysis curves of workflows in Grayson et al. 2023.



Conclusions

- Primary reason: Labor required to package and explain
- Workflows and environments help, but not everything
- Average lifetime is only a few years
- Linking problems could benefit to sem. web

Technical approaches to automatic reproducibility



1. Background
2. Characterization of reproducibility in practice
3. **Technical approaches to automatic reproducibility**
 1. Prior approaches
 2. Computational provenance
 3. Review of sys-level prov tracers
 4. PROBE
4. Proposed work
5. Conclusion

Prior approaches

- Package managers, e.g., Nix, Spack
- Virtualization, e.g., Docker
- Sys-level provenance, e.g., RR-debugger

Package managers

*It is pretty easy to be confused.
Especially since it is also a language
yet all the other replies at the time
have writing have failed to mention
that.*

*Conceptually I'm having a hard time
wrapping my head around it since
package managers are so tightly tied
to an OS... I feel like surely I can't just
use it in place of (say) apt or pacman*

Virtualization (build script)

1. Distribute VM/container build script
2. Tradeoff between reproducibility (exact tag) and availability (loose tag)
3. Is the second line `apt-get update`?
4. 55% of dependency specs incomplete ([Shaffer et al. 2021](#))
5. Of those, 50% of were non-reproducible in 10 days ([Shaffer et al. 2021](#))
6. This method can work, but it needs a repro-ible package manager

Computational provenance

1. Process by which a file was created
2. Inputs to that process
3. Provenance of those inputs

- Handles missing environment
- Handles missing data
- Even username comes from files
- Similar to workflows, but "observed" deps rather than "reported"

Review of sys-level prov tracers

Grayson, Aguilar, Milewicz, Katz, Marinov. *A benchmark suite and performance analysis of user-space provenance collectors*. ACM REP '24

[10.1145/3641525.3663627](https://doi.org/10.1145/3641525.3663627)

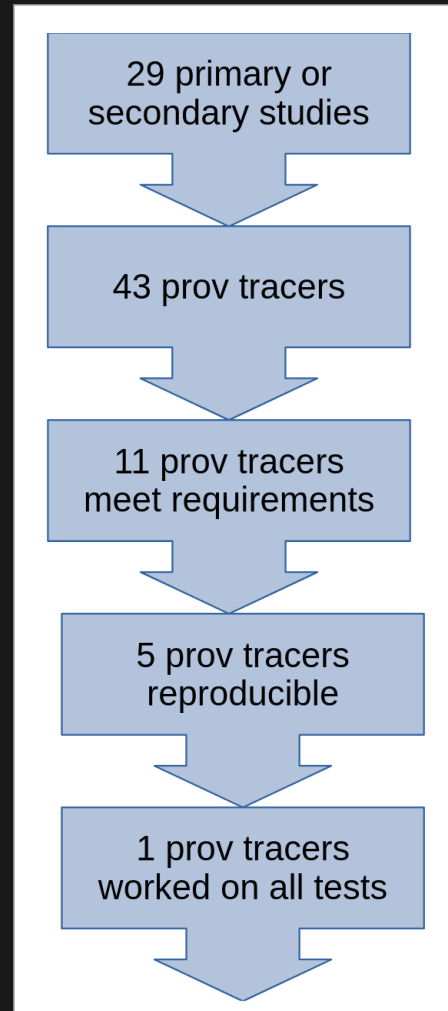
Search queries: "Computational provenance" &
"System-level provenance"

Feasibility study

Benchmarking

Linux sys-level prov tracers

Strategy	Modified kernel?	Privileges required?	Provenance tracer
Modify kernel	yes	yes	PASSv2, CamFlow, ...
Built-in auditing	no	yes	SPADE, eBPF, ...
User tracing	no	no	Sciunit, ReproZip, ...



Performance is critical to usability

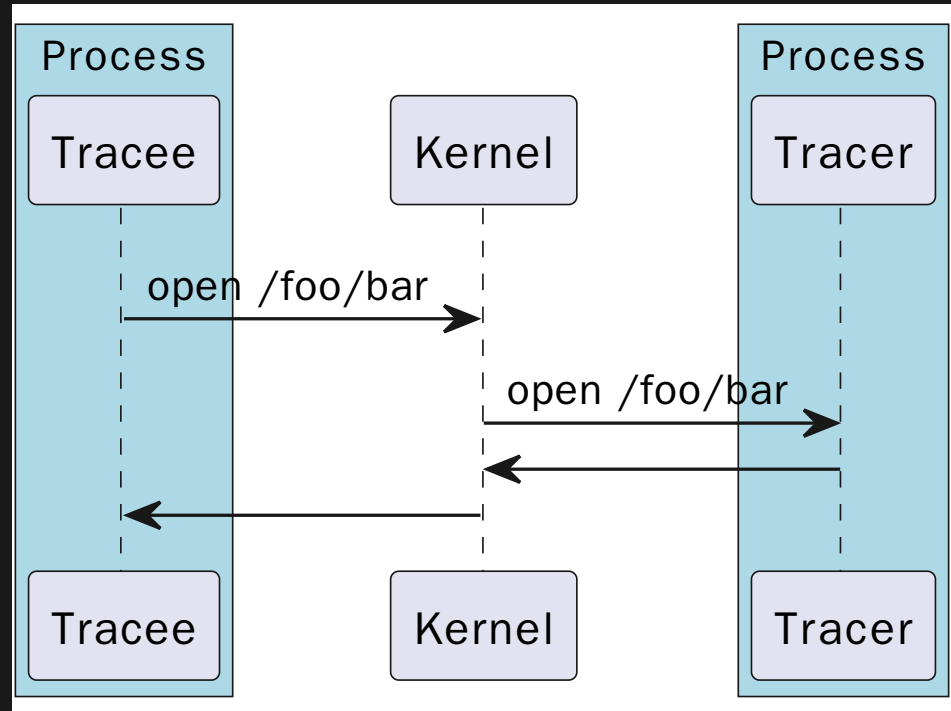
Measure "percent overhead". 100% means runtime $x \rightarrow 2x$ with prov

ReproZip avg overhead was 193%

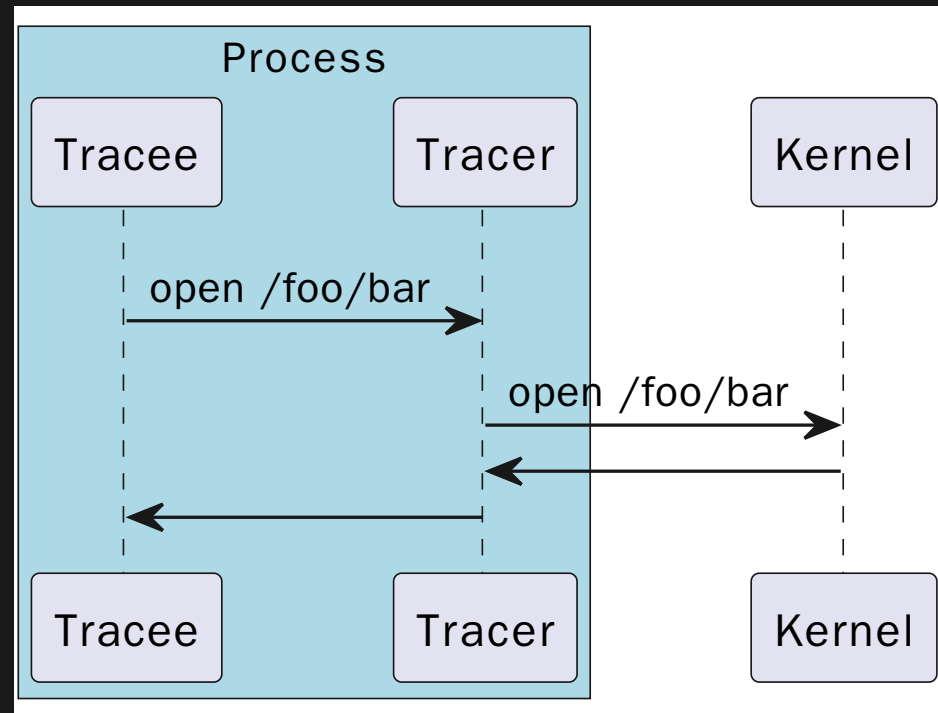
Performance

Benchmark	fsatrace	strace	ReproZip
BLAST	0	2	8
Tar Unarchive	4	114	149
Python import	5	84	346
VCS checkout	5	160	428
Postmark	2	650	1733
cp	37	380	5791
Others not shown	...	...	...
Geometric mean	0	66	193

ptrace



Library interposition



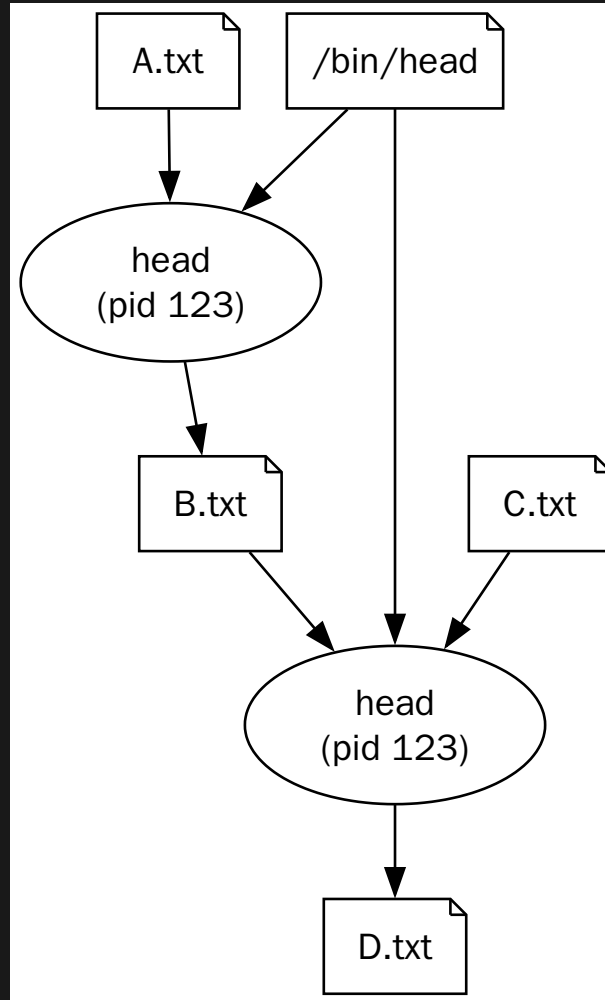
Talk

Grayson, Milewicz, Katz, Marinov. *PROBRSE:
Provenance Replay/Observation Engine for Research
Software Engineers*. US-RSE '24
[10.5281/zenodo.13963643](https://doi.org/10.5281/zenodo.13963643)

PROBE

- Speed
- Reuse environment for new exe
- ptrace → library interposition

```
$ probe record ./script.sh  
$ probe export dataflow-graph
```



```
$ probe record ./script.sh
hello world
$ probe export docker-image test:1.0.0
$ docker run --rm test:1.0.0
hello world
```

(Planned feature)

```
$ probe record ./script.sh  
hello world
```

```
$ probe export spack-env  
$ # Send spack-env.tar.gz to your friends
```

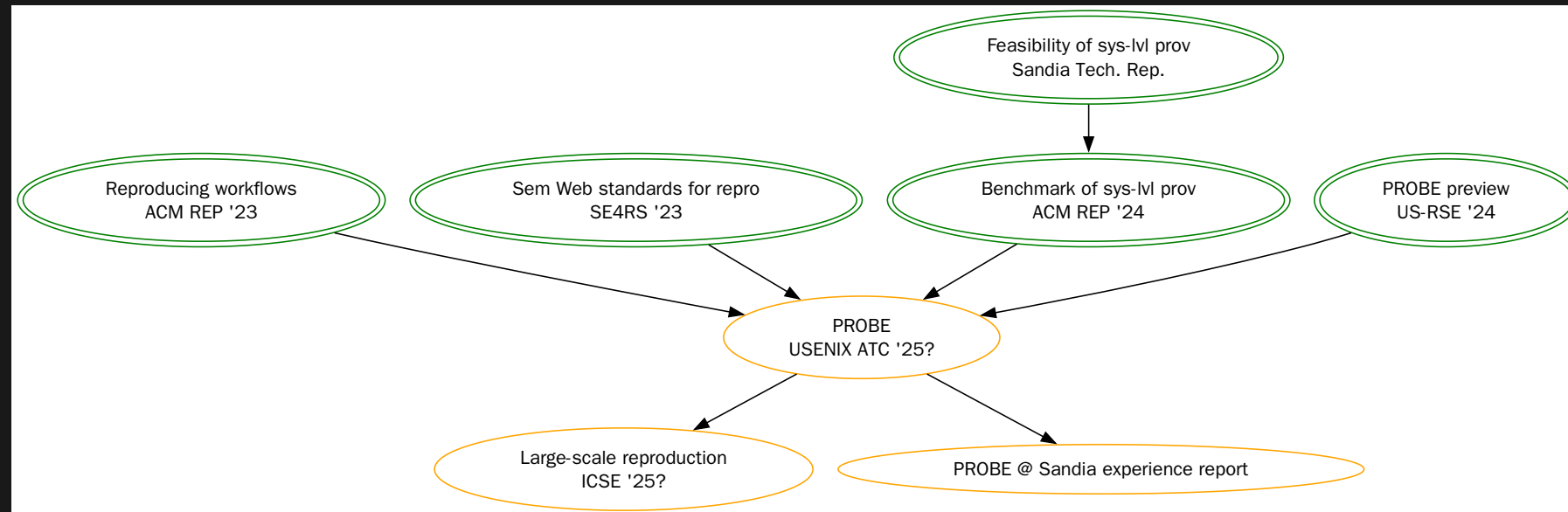
```
$ tar xvf spack-env.tar.gz  
$ cd spack-env  
$ ./script.sh  
hello world
```

Conclusion

1. System-level provenance tracing can address missing dependencies/data
2. Existing system-level provenance tracers require root or are too slow
3. Library interposition may be faster than traditional tracing

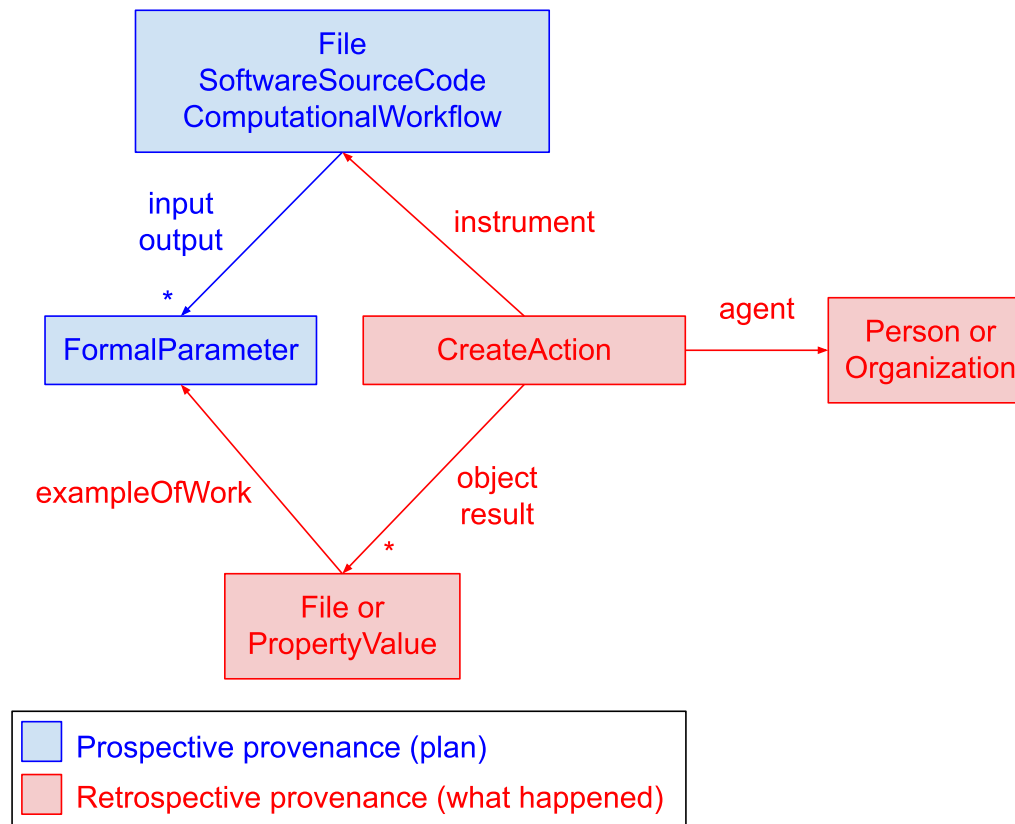
Proposed work





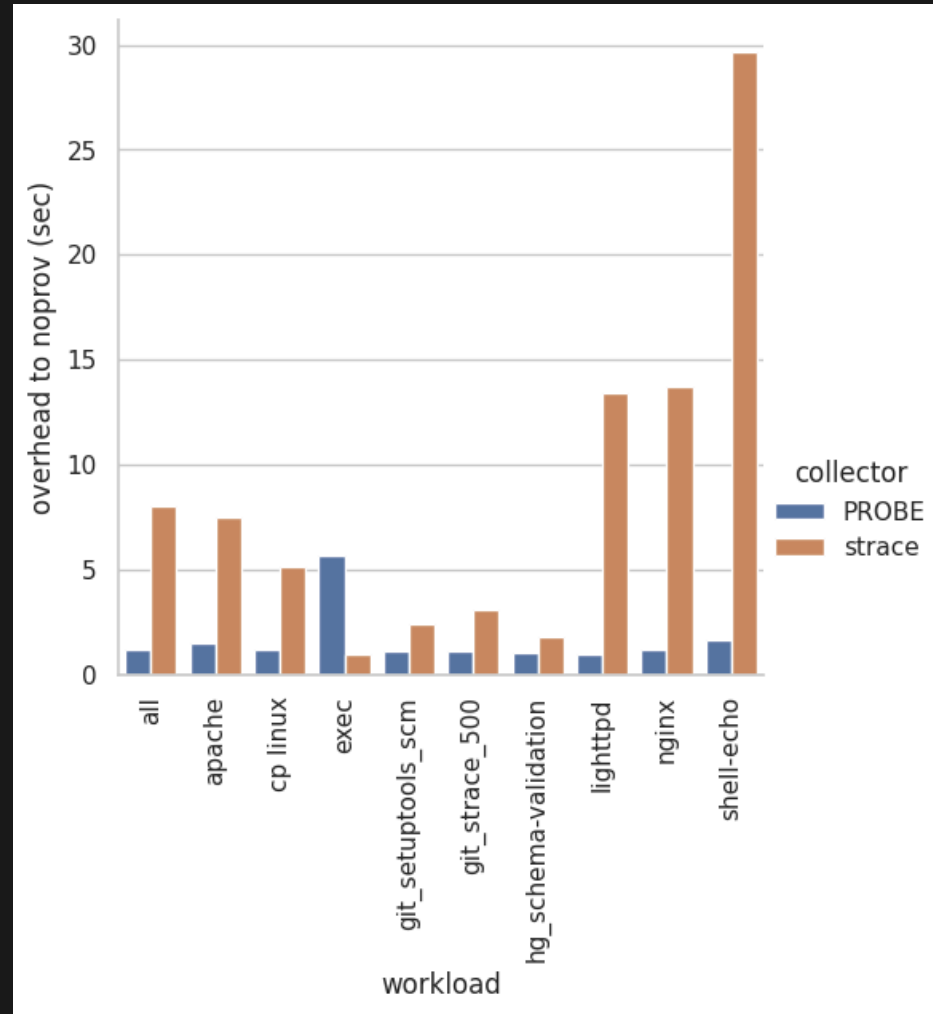
PROBE: Provenance Replay OBservation Engine

- Improve robustness of PROBE
- Performance analysis on benchmark suite
- Workflow Run RO-Crate (WRROC), (uses RO-terms ont.) [Leo et al. 2024](#)



System-level provenance tracing

- Library interp. vs user-level tracing
- Evaluate robustness c.f. other methods
- Storage cost for daily use?
- Use interoperable standards



Reproduction of Collberg et al.

- Original study looks at 11 CS journal/conf 3 years later; tries to "build environment" for those papers
- Has the status of the artifacts changed? → estimate decay rate
- Same procedure on new data yield different answer? → improvements
- Can PROBE encapsulate dependencies in real-world cases? at what cost?

PROBE @ Sandia experience

- Unforeseen system integration issues
- Storage and performance cost in real-world env
- User test with think-aloud protocol

Schedule

- 2025 Dec 01: stabilize PROBE's core functionality
- 2025 Dec 20: stabilize reproduced C&P dataset
- **2025 Jan 14**: USENIX ATC '25 submission deadline (PROBE)
- **2025 Mar 22 (expected)**: ICSE '25 submission deadline (reproduce C&P)
- **2025 May 28 (expected)**: eScience '25 submission deadline (PROBE @ Sandia)

Conclusion



jonrawlinson.com

Takeaways

1. $\exists$ valuable but unexplored technical approaches to improve reproducibility
2. It is feasible to research those for my 12-month thesis.

Future work

- Use environment manipulation (from provenance) for automatic repair
- User-testing of repro tools
- Sem web integration

Backup slides

$$f_h(x, c) = (y, c')$$

- f : run function
- x : user input (measurement procedure)
- y : user output (measurement)
- h : hardware and kernel (measurment system)
- c, c' : configuration function: $\mathbb{C} \rightarrow \mathbb{A}$ (operating conditions)